

IAsLab ORCHID: Plataforma de Orquestación y Gobernanza de Cargas de IA/ML en la Infraestructura de la Universidad Icesi

PROYECTO DE GRADO

Presentado como parte de los requisitos para obtener los títulos de
Ingeniero Telemático e Ingeniero de Sistemas

por

JUAN JOSÉ DE LA PAVA GIRALDO
Ingeniería Telemática e Ingeniería de Sistemas

JUAN CAMILO MELO LÓPEZ
Ingeniería Telemática

JUAN DAVID PACHECO VARGAS
Ingeniería Telemática

Tutor: Kevin David Rodríguez Belalcázar
Profesor acompañante: Alejandro Muñoz Bravo

Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas

Universidad Icesi
2026

Índice general

Resumen	v
Abstract	vi
Lista de acrónimos	vii
Glosario de términos	viii
1 Motivación y antecedentes	1
1.1 Contexto	1
1.2 Antecedentes del problema	2
1.3 Justificación	3
2 Descripción del problema	5
2.1 Identificación del problema	5
2.2 Formulación del problema	6
3 Objetivos	7
3.1 Objetivo general	7
3.2 Objetivos específicos	7
4 Marco teórico	8
4.1 El ciclo de vida de las cargas de inteligencia artificial	8
4.2 Despliegue e inferencia de modelos	9
4.3 Contenedores y orquestación de cargas de cómputo	10
4.4 Extensibilidad declarativa del orquestador	11
4.5 Gobernanza de recursos compartidos	12
4.6 Identidad federada y control de acceso	13
4.7 Observabilidad y telemetría de infraestructura acelerada	13
4.8 Marco conceptual y categorías de análisis	14
5 Estado del arte	16
6 Metodología	17
6.1 Estrategia metodológica	17

6.2	Fases de desarrollo del proyecto	18
6.3	Cronograma	20
6.4	Análisis de riesgos y limitaciones	22
7	Contribución y resultados del proyecto de grado	24
7.1	Aportes relacionados con el objeto del proyecto	24
7.2	Aportes relacionados con el desarrollo de capacidades del investigador	25
7.3	Resultados y entregables	25

Índice de tablas

4.1	Definición operativa de los conceptos del marco teórico y objetivo específico que los emplea.	14
6.1	Fases de desarrollo del proyecto, su correspondencia con los objetivos específicos y su duración.	19
6.2	Cronograma del proyecto, por <i>sprints</i> de tres semanas, con entregables asociados. . .	20
7.1	Trazabilidad entre objetivos específicos, resultados esperados y entregables.	25

Índice de figuras

Resumen

Palabras clave:

Abstract

Keywords:

Lista de acrónimos

CPU	Unidad central de procesamiento (<i>Central Processing Unit</i>)
DCGM	Gestor de GPU para centros de datos, de NVIDIA (<i>Data Center GPU Manager</i>)
GGUF	Formato de archivo para modelos cuantizados empleado por <i>llama.cpp</i> y Ollama
GPU	Unidad de procesamiento gráfico (<i>Graphics Processing Unit</i>)
IAsLab	Laboratorio de Inteligencia Artificial y Arquitectura de Software de la Universidad Icesi
LLM	Modelo de lenguaje de gran escala (<i>Large Language Model</i>)
MLOps	Operaciones de aprendizaje automático (<i>Machine Learning Operations</i>)
PDG	Proyecto de grado
RBAC	Control de acceso basado en roles (<i>Role-Based Access Control</i>)
SSO	Inicio de sesión único (<i>Single Sign-On</i>)
SUS	Escala de usabilidad del sistema (<i>System Usability Scale</i>)
VRAM	Memoria de acceso aleatorio de video (<i>Video Random Access Memory</i>)

Glosario de términos

SAAMFI Servicio institucional de identidad de la Universidad Icesi. Funciona exclusivamente como proveedor de identidad (*Identity Provider*), en el mismo sentido que Keycloak o Auth0: registra a los usuarios y su esquema de roles y permisos, y emite un *token* que cada sistema integrado consume según sus propias reglas. No impone políticas de autorización sobre los sistemas que lo usan, de modo que la traducción de un rol en un límite de consumo de *hardware* corresponde a la plataforma desarrollada en este proyecto.

Capítulo 1

Motivación y antecedentes

1.1. Contexto

Llevar un modelo de aprendizaje automático a producción exige más que entrenarlo. Sculley et al. (2015) señalan que su código es solo una pequeña fracción de un sistema real, rodeado de una infraestructura extensa que incluye el servicio y el monitoreo de los modelos. Kreuzberger et al. (2023) definen las operaciones de aprendizaje automático (MLOps) como un paradigma que abarca la conceptualización, implementación, monitoreo, despliegue y escalabilidad de estos productos, e incluyen un componente dedicado a vigilar de forma continua el desempeño en servicio y el estado de la infraestructura. Aun así, Eken et al. (2026) advierten que un cuerpo de conocimiento integrado sobre MLOps sigue sin lograrse, porque su alcance es amplio y abarca desafíos muy diversos de esa transición.

La infraestructura mencionada exige, además, una capacidad de procesamiento creciente. Desde la llegada del aprendizaje profundo a comienzos de la década de 2010, el cómputo necesario para entrenar los sistemas más avanzados se ha duplicado aproximadamente cada seis meses (Sevilla et al., 2022). La industria domina una porción cada vez mayor de ese poder de cálculo, mientras la academia cuenta con menos del que necesita para investigar, sobre todo en unidades de procesamiento gráfico (GPU) (Ahmed et al., 2023). En ese escenario, las GPU que ya posee una universidad adquieren un valor estratégico.

Cuando ese *hardware* se comparte, optimizar su uso se vuelve un reto estructural. Incluso en plataformas empresariales multiusuario de aprendizaje profundo, algunos trabajos presentan una utilización notablemente baja de las GPU asignadas, lo que desperdicia recursos valiosos y reduce la productividad del desarrollo (Gao et al., 2024). El estudio analizó 400 casos reales en un entorno corporativo de gran tamaño, elegidos entre los que aprovechaban en promedio la mitad o menos de su capacidad, e identificó en ellos 706 problemas de este tipo, originados en el código de cada trabajo por cómputo insuficiente en el acelerador o por interrupciones de tareas ajenas a él. Si el problema aparece a esa escala, es razonable que un laboratorio académico también necesite herramientas para detectarlo y controlarlo.

Ese reto se presenta, en una dimensión más modesta, en los equipos de cómputo que el Laboratorio de Inteligencia Artificial y Arquitectura de Software (IAsLab) de la Universidad Icesi destina a la ciencia de datos y la inteligencia artificial. Este proyecto de grado se enmarca en el macroproyecto *Ecosistema computacional de servicios de software para el desarrollo de proyectos del IAsLab para Industria 4.0 y e-Salud en el contexto de la Transformación Digital*, cuyo propósito es reducir la brecha entre la investigación aplicada del IAsLab y su adopción en entornos productivos. Para ello, la iniciativa busca consolidar una base común de componentes y servicios de *software* reutilizables y estandarizados que eleven la madurez tecnológica de sus desarrollos y faciliten su paso de la experimentación académica a soluciones robustas y seguras, acordes con las exigencias de los sectores industrial y clínico.

En ese marco, un proyecto de grado anterior construyó un sistema web que actúa como plano de control (*control plane*) del entrenamiento distribuido. Esa herramienta centraliza la configuración, ejecución y seguimiento de los entrenamientos, aprovisiona los nodos de procesamiento e integra SAAMFI como proveedor institucional de identidad. La presente propuesta aporta al propósito común la etapa siguiente del ciclo, pues construye la plataforma que despliega y sirve los modelos resultantes, gobierna el uso compartido de las GPU y observa el estado del *hardware*, sobre esos mismos nodos.

Alrededor de esa infraestructura intervienen actores con necesidades distintas. Los estudiantes y profesores de la universidad requieren GPU para sus proyectos e investigaciones, y se identifican mediante sus roles institucionales. Los administradores del IAsLab operan los equipos y asumen su soporte. Quienes usan el plano de control anterior comparten esas máquinas, que además deben seguir disponibles para las clases.

1.2. Antecedentes del problema

[TODO: Cuidado con el lenguaje, mantienen un lenguaje formal previo a este capítulo y empieza este mencionando .^a un máximo de 4 o 5 personas de su círculo, o no entregaba nada. Esta información es falsa, los estudiantes sí tenían acceso a través de la VPN, (ej curso ingesoft 4) o participantes del laboratorio IasLab, si un estudiante o profesor solicitaba acceso se le daba, no obstante el proceso era manual y en ocasiones tomaba un tiempo extenso para aceptarse la solicitud, entonces no es que toda la comunidad no pudiera acceder y era monopolizado por un conjunto de personas.] Antes del proyecto de grado previo, el IAsLab no tenía una política formal de acceso a sus equipos. El acceso era binario y manual, pues el laboratorio entregaba credenciales de VPN y de administrador a título individual, a un máximo de cuatro o cinco personas de su círculo, o no entregaba nada. El resto de la comunidad estudiantil recurría a servicios externos como Google Colab o prescindía de cómputo especializado, lo que produjo una subutilización sostenida del *hardware*. A la fecha, el laboratorio usa sus equipos de forma experimental y todavía no en explotación continua.

[TODO: La conclusión no se deriva directamente de la premisa: se pasa de afirmar que el proceso era manual a concluir que solo quienes tenían acceso a los equipos podían hacerlo, sin explicar la relación entre ambas ideas.] Poner un modelo en operación también

era un proceso manual. Antes de este proyecto, el laboratorio desplegaba modelos ejecutando a mano *scripts* de Python que instalaban los motores de inferencia y sus dependencias, sin ninguna plataforma que automatizara el proceso, de modo que solo quienes tenían acceso a los equipos podían hacerlo. El proyecto de grado previo automatizó el aprovisionamiento de nodos para el entrenamiento distribuido, pero no abordó las etapas posteriores al entrenamiento. Hoy ninguna plataforma del laboratorio permite desplegar y servir modelos ya entrenados ni gobernar el uso compartido de las GPU, y los administradores no pueden observar en tiempo real el consumo exacto de la infraestructura.

Las universidades que comparten clústeres de GPU enfrentan problemas semejantes. Xu et al. (2025), a partir de la operación de un clúster de más de 160 GPU al servicio de más de 480 usuarios de su universidad, señalan que compartir estos recursos es esencial para aprovecharlos y ampliar el acceso, aunque gestionarlos plantea desafíos que van desde la configuración del sistema hasta el reparto equitativo entre usuarios, con personal limitado para operarlos. Según los autores, muchas universidades adoptan una herramienta conocida con configuraciones subóptimas y terminan subutilizando recursos ya limitados. Un esquema simple y común, que entrega a los usuarios acceso directo por consola a las máquinas con GPU, carece de mecanismos de aislamiento y de gestión, obliga a cada usuario a configurar su entorno y a coordinar a mano el uso de las GPU, y dificulta que el reparto sea equitativo.

En una plataforma académica de mayor escala, un mecanismo de gobernanza tuvo efectos medibles. En la National Research Platform, un clúster Kubernetes multiusuario que en 2024 atendió a investigadores de 50 campus, la introducción de un sistema de reservas para las GPU de mayor demanda elevó la utilización promedio de las GPU del clúster del 21.49 % al 31.37 %, aunque sus operadores reconocen que aún hay margen de mejora (Weitzel et al., 2025). Estos antecedentes coinciden con la situación del IAsLab, donde el acceso directo y manual a los equipos no ofrece un reparto controlado de las GPU ni visibilidad sobre su uso, condiciones que exponen la infraestructura a un riesgo de monopolización a medida que crece el número de usuarios.

1.3. Justificación

Construir esta plataforma es pertinente porque responde a necesidades concretas del IAsLab. El laboratorio despliega hoy sus modelos de forma manual, no cuenta con reglas que repartan sus GPU entre los usuarios y no dispone de una vista del consumo de su infraestructura, mientras crece la comunidad que podría aprovecharla. Una solución que automatice el despliegue, gobierne el acceso mediante cupos y observe el estado del *hardware* sustituye la entrega manual de credenciales por un servicio al que estudiantes y profesores acceden de forma controlada.

Ese cambio amplía el acceso al cómputo especializado. Al reemplazar el uso restringido a unas pocas personas por un servicio con reglas de uso explícitas, la capacidad instalada queda al alcance de más estudiantes y profesores, que dejan de depender de suscripciones comerciales que no todos pueden costear, y se fortalecen otros proyectos que requieren esos recursos. La universidad, por su parte, obtiene un mayor retorno de la inversión que ya hizo en equipos, al usarlos en lugar de adquirir créditos en nubes públicas.

Ese mejor aprovechamiento tiene, además, un efecto ambiental. En un clúster universitario compartido, el consumo base de los equipos, que incluye la potencia en reposo de sus componentes, se mantiene relativamente constante sin importar cuántas GPU estén ocupadas, y la GPU representa la mayor parte de la potencia demandada (Xu et al., 2025). Un acelerador asignado que no se usa gasta energía sin producir resultados, de modo que gobernar su reparto y detectar la subutilización favorece un uso más eficiente de la energía del laboratorio.

Por último, el proyecto es relevante porque su escenario reúne características que lo distinguen de los entornos industriales de gran escala. Un laboratorio universitario con pocos nodos de GPU usa las mismas máquinas para las clases y para la investigación, y necesita gobernar el servicio de modelos y observar el estado del *hardware* sobre infraestructura propia. Documentar el diseño, la operación y la evaluación de una plataforma en esas condiciones genera una experiencia transferible a otras instituciones académicas con recursos limitados.

Capítulo 2

Descripción del problema

2.1. Identificación del problema

[TODO: La estructura general del apartado es coherente y sigue una secuencia problema → síntomas → causas → efectos. Sin embargo, la relación causal entre estos elementos no siempre es explícita: algunas “causas secundarias” parecen más bien limitaciones o condiciones del contexto, mientras que los “efectos secundarios” provienen de problemas distintos (acceso, gobernanza y monitoreo). Conviene revisar que cada causa explique directamente un síntoma y que cada efecto se derive claramente del problema o de sus síntomas.] [TODO: .^{El} problema central es que....^{es} un lenguaje muy básico. La formulación del problema resulta demasiado coloquial; convendría emplear una construcción más académica que introduzca la situación problemática.]

El problema central es que la infraestructura de GPU del IAsLab no puede aprovecharse de forma compartida, controlada y observable por la comunidad académica que la necesita. Ese problema se manifiesta en tres síntomas. Poner un modelo en operación exige un procedimiento manual, el uso de los equipos depende de credenciales que se entregan a pocas personas, y los administradores no pueden observar en tiempo real el consumo ni el estado de los nodos.

[TODO: "La causa principal es que..."] La causa principal es que la operación de la infraestructura descansa en procedimientos manuales, porque no existe una capa de *software* que automatice el despliegue de modelos, reparta los recursos entre usuarios y registre su uso. A ella se suman tres causas secundarias. El proyecto de grado previo automatizó el aprovisionamiento de nodos para el entrenamiento, pero no las etapas posteriores. SAAMFI identifica a los usuarios y sus roles, sin traducir esos roles en límites de consumo. Por último, los nodos con GPU también se usan para las clases sin que exista una vista de su ocupación.

El efecto principal es la subutilización de una infraestructura en la que la universidad ya invirtió, pues la mayoría de los estudiantes, sin acceso a los equipos, recurre a servicios externos o prescinde del cómputo especializado. Los efectos secundarios recaen sobre distintos involucrados. Los administradores concentran la carga de habilitar accesos y desplegar modelos, y los estudiantes y profesores carecen de un servicio para consumir los modelos que entrenan. Si el acceso se amplía

sin reglas de reparto, aumenta además el riesgo de que unos pocos usuarios o trabajos monopolicen las GPU, y de que fallas como el congelamiento de un nodo por desbordamiento de memoria pasen inadvertidas hasta afectar a otros usuarios.

2.2. Formulación del problema

[TODO: La formulación es coherente con el planteamiento general, pero la pregunta incorpora directamente la solución propuesta. Revisar si se busca formular una pregunta de investigación sobre el problema o una pregunta de desarrollo tecnológico orientada a construir la plataforma. También conviene mantener consistencia entre los conceptos de roles, cupos y reglas de reparto] La infraestructura de GPU del IAsLab no puede aprovecharse de forma compartida, controlada y observable, porque su operación depende de procedimientos manuales y ninguna herramienta automatiza el despliegue de modelos, reparte los recursos según los roles de los usuarios ni registra el estado y el consumo de los nodos. Como consecuencia, la capacidad instalada se subutiliza, la mayoría de los estudiantes queda sin acceso a ella, el ciclo de MLOps se interrumpe después del entrenamiento, y ampliar el uso sin reglas expondría las GPU a un reparto inequitativo y a fallas que nadie detecta a tiempo. De ello se deriva la pregunta del proyecto. ¿Cómo construir, sobre la infraestructura del IAsLab, una plataforma de despliegue e inferencia de modelos, reserva de cupos con reglas de reparto y observabilidad avanzada, sin comprometer la estabilidad ni la disponibilidad del *hardware*? Responderla consolida en una sola plataforma las etapas hoy desarticuladas de despliegue, gobernanza y monitoreo.

Capítulo 3

Objetivos

3.1. Objetivo general

Construir la plataforma de orquestación y gobernanza de cargas de inteligencia artificial y aprendizaje automático del IAsLab, mediante módulos de despliegue de inferencia, gobernanza de recursos por rol y observabilidad de infraestructura, para consolidar el ciclo de vida de MLOps *on-premise* en la Universidad Icesi.

3.2. Objetivos específicos

1. Desarrollar un módulo de observabilidad de la plataforma para la captura y visualización de métricas de *hardware* y de carga de trabajo de los nodos de cómputo del IAsLab, orientado a la supervisión del estado de dichos nodos.
2. Implementar un sistema de cuotas y restricciones de infraestructura, acoplado a los roles provistos por SAAMFI y a un rol de administrador de la plataforma, que reserve cupos de cómputo para las cargas de inferencia con prioridad de los profesores sobre la asignación de la capacidad de la sala, con el fin de prevenir la monopolización de las GPU.
3. Implementar sobre el clúster de cómputo del IAsLab el despliegue concurrente de modelos de inteligencia artificial ya entrenados, mediante motores de inferencia empaquetados como imágenes de contenedor, con el fin de llevar los modelos del laboratorio a una fase de servicio consumible.
4. Evaluar el desempeño y la aceptación de la plataforma mediante pruebas de carga con veinte usuarios simultáneos y la aplicación del *System Usability Scale* a usuarios del laboratorio, con el fin de determinar si la plataforma sostiene esa concurrencia y en qué banda de referencia publicada se ubica su usabilidad percibida.

Capítulo 4

Marco teórico

[TODO: Me gusta mucho como abordan los conceptos teóricos, no obstante este documento hasta donde va no proporciona ni una sola imagen, es poco ilustrativo y es agotador leerlo aunque muestra información relevante, este marco teórico abre la posibilidad (solo es una posibilidad) de integrar imágenes, figuras, gráficos o demás material de apoyo que ayude al lector aclarar conceptos y términos, usualmente el marco teórico es tedioso de leer porque se requiere de un nivel técnico alto para contextualizarse. Si deciden hacerlo, lo cual aconsejo, no lo hagan de manera indiscriminada (en cada sección) sino que miren en qué lugares aporta información. Nota: Si ponen imágenes o figuras, gráficos, deben de referenciarlas y explicarlas textualmente: **.En la figura X se presenta.....**"] Construir una plataforma que despliegue modelos de inteligencia artificial sobre *hardware* compartido, gobierne quién los ejecuta y observe lo que ocurre en la infraestructura mientras lo hacen exige integrar conocimiento de cinco dominios de la ingeniería de sistemas, que abarcan el ciclo de vida de los sistemas de aprendizaje automático, el servicio de modelos en producción, la orquestación de contenedores y su modelo de extensión, la gobernanza de recursos compartidos entre usuarios con derechos desiguales, y la observabilidad de infraestructura acelerada. Este capítulo los recorre de lo general a lo particular, desde la fase del ciclo de vida que el proyecto ocupa hasta las técnicas concretas que la instrumentan, y cierra con las definiciones operativas y las categorías bajo las cuales se analiza el resultado.

4.1. El ciclo de vida de las cargas de inteligencia artificial

MLOps designa un paradigma que reúne buenas prácticas, conceptos y una cultura de desarrollo para la conceptualización, implementación, monitoreo, despliegue y escalabilidad de extremo a extremo de los productos de aprendizaje automático, y que se apoya en tres disciplinas, el aprendizaje automático, la ingeniería de *software* (en particular DevOps) y la ingeniería de datos (Kreuzberger et al., 2023). El enfoque surgió para reducir el esfuerzo y mejorar la integración entre quienes llevan los modelos al entorno de producción (Lima et al., 2022). Sculley et al. (2015) explican por qué esa integración pesa tanto. En un sistema de aprendizaje automático real, el código del modelo ocupa

una fracción pequeña del conjunto, y la infraestructura que lo rodea, que incluye la configuración, la infraestructura de servicio, la gestión de los recursos de cómputo y el monitoreo, es extensa y compleja. Los mismos autores advierten que estos sistemas tienen una capacidad especial para acumular deuda técnica, difícil de detectar porque se sitúa en el nivel del sistema completo, fuera del código del modelo.

Dentro de ese ciclo, la literatura separa dos fases cuyos objetivos de diseño se oponen. El entrenamiento consume datos para producir un artefacto de pesos, maximiza el *throughput* sobre un horizonte largo y no interactivo, tolera la interrupción y se mide por el tiempo hasta converger. El servicio de modelos parte de un artefacto ya producido, atiende solicitudes individuales en tiempo real y se mide por latencia y por estabilidad bajo carga variable (Muiruri et al., 2026). La distinción se aplica a cualquier tipo de modelo. Un clasificador clásico, un modelo de visión por computador y un modelo generativo de gran escala atraviesan el mismo ciclo y presentan la misma separación entre la fase que produce el artefacto y la fase que lo pone en servicio. Lo que cambia entre ellos es el perfil de consumo y el conjunto de métricas relevantes, sin que la estructura del ciclo se altere. El proyecto se ubica por completo en la fase de servicio, para cargas de inteligencia artificial en general. El entrenamiento corresponde al proyecto hermano del mismo macroproyecto y queda fuera del alcance de esta plataforma, que no lo ejecuta, no lo gobierna ni lo clasifica.

4.2. Despliegue e inferencia de modelos

Desplegar un modelo consiste en instalar un artefacto ya entrenado en un entorno de ejecución donde pueda recibir solicitudes de forma sostenida, e inferir es la operación por la cual ese artefacto produce una salida a partir de una entrada nueva. Entre ambos se interpone el motor de inferencia, componente que carga los pesos en la memoria del acelerador, gestiona el conjunto de solicitudes concurrentes y expone una interfaz de red por la que los clientes consumen el modelo. Muiruri et al. (2026) lo caracterizan como el punto donde se concentran las decisiones de rendimiento del sistema en producción, porque condiciona la latencia percibida, la concurrencia que un nodo sostiene y la memoria que un modelo ocupa realmente. De esa caracterización se derivan las métricas con que se evalúa el servicio, que son el tiempo hasta el primer *token*, la latencia entre *tokens*, el *throughput* agregado y el tiempo de espera en cola. Las cuatro guardan dependencias entre sí, porque aumentar el tamaño del lote eleva el *throughput* agregado y a la vez degrada la latencia entre *tokens* de cada solicitud, de modo que el motor opera siempre sobre un compromiso explícito.

Cuando el modelo en servicio es un modelo de lenguaje de gran escala, el motor enfrenta un cuello de botella particular, porque la caché de atención crece con la longitud de cada petición y, bajo solicitudes concurrentes de longitud variable, fragmenta la memoria del acelerador hasta desperdiciar una fracción sustancial de la capacidad instalada. Kwon et al. (2023) resuelven esa fragmentación paginando la caché en bloques de tamaño fijo, con una analogía directa a la memoria virtual de un sistema operativo. Yu et al. (2022) atacan el mismo problema desde la programación de peticiones y muestran que planificar a nivel de iteración, en lugar de esperar a que el lote completo termine, eleva de forma sustancial el *throughput* agregado del servidor. Miao et al. (2025) organizan estas técnicas, junto con la compresión de modelos y la programación de *kernels*, como un espectro continuo de optimización que abarca desde el algoritmo hasta el sistema.

Dentro de ese espectro, la cuantización posentrenamiento ocupa un lugar particular. La técnica reduce la precisión numérica con la que se representan los pesos, con una degradación de calidad acotada y medida experimentalmente (Frantar et al., 2023; Lin et al., 2024; Dettmers et al., 2022; Chen et al., 2026). Para este proyecto su relevancia está en la capacidad que libera, porque explica por qué modelos de 7 000 a 14 000 millones de parámetros caben dentro de los 24 GB de memoria de video de un nodo con tarjeta RTX 4090, y por tanto por qué el laboratorio puede ponerlos en servicio sin recurrir a inferencia distribuida entre nodos. La plataforma despliega pesos que ya llegan cuantizados, o se apoya en la cuantización que el propio motor aplica al cargar el modelo, como hace *llama.cpp*, sin ejecutar ningún proceso propio de cuantización.

Los motores disponibles ocupan puntos distintos de ese espectro, y esa diversidad es la razón teórica por la cual la plataforma no puede acoplarse a ninguno. vLLM aplica paginación de caché y programación por iteración para maximizar el *throughput* bajo concurrencia alta, y paga ese *throughput* con mayor latencia entre *tokens* cuando el lote crece (Kwon et al., 2023; Miao et al., 2025). *llama.cpp* adopta el compromiso contrario y prioriza la latencia de un único flujo junto con un *footprint* de despliegue mínimo, lo que lo hace adecuado para sesiones individuales sobre *hardware* de clase estación de trabajo. Otros motores atienden familias de modelos que no son de lenguaje, con formatos de artefacto y perfiles de cómputo propios. La literatura revisada no ofrece una comparación controlada entre estas opciones sobre el tipo de *hardware* que el IAsLab tiene instalado, de modo que cualquier preferencia operativa del laboratorio constituye una decisión de ingeniería propia, pendiente de respaldo experimental. Esa ausencia obliga al proyecto a tratar el motor como pieza intercambiable y a producir la comparación mediante medición.

4.3. Contenedores y orquestación de cargas de cómputo

Un motor de inferencia, con sus pesos, sus dependencias de CUDA y sus versiones concretas de bibliotecas, constituye una unidad de *software* cuya reproducción por instalación manual es frágil. El contenedor la empaqueta junto con su entorno de ejecución completo en una imagen inmutable, de modo que lo que se ejecuta en un nodo es idéntico a lo que se probó en otro; el orquestador resuelve el problema siguiente, que consiste en decidir en qué máquina se ejecuta cada unidad, reiniciarla cuando falla y exponerla a la red de forma estable. Kubernetes se consolidó como el orquestador de referencia para esa tarea. Un análisis bibliométrico identifica la programación de recursos, el cómputo distribuido y la inteligencia artificial entre sus focos de investigación más activos (Carrión, 2022), y una revisión multivocal documenta, junto a los beneficios que los profesionales reportan, la carencia de herramientas nativas de diagnóstico entre sus obstáculos recurrentes (Shamim et al., 2022). Ese balance sostiene dos decisiones del proyecto a la vez, que son adoptar Kubernetes como sustrato en lugar de construir un programador de clúster propio, y prever desde el diseño la capa de observabilidad que cubra el vacío de diagnóstico que la propia literatura señala.

Las primitivas que el orquestador ofrece se aplican de forma directa al caso del laboratorio. Los selectores de nodo, junto con las marcas y tolerancias, dirigen las cargas que requieren aceleración a las máquinas con tarjeta gráfica y reservan las demás para los servicios de control. Los espacios de nombres dividen el clúster en fronteras lógicas de administración sobre las que se aplican límites agregados de consumo. Los volúmenes persistentes desacoplan el almacenamiento de los pesos del

ciclo de vida de los contenedores que los sirven, lo que evita descargar decenas de gigabytes cada vez que un despliegue se reinicia. Estos mecanismos se aplican a cualquier tipo de carga, y esa generalidad permite gobernar con las mismas herramientas los distintos modelos en servicio y los servicios auxiliares de la plataforma.

4.4. Extensibilidad declarativa del orquestador

Kubernetes no fue diseñado para cargas de inteligencia artificial y sin embargo las hospeda sin que su núcleo haya sido rediseñado. El mecanismo que lo permite es su modelo declarativo. Burns et al. (2016), a partir de una década operando los sistemas Borg, Omega y Kubernetes, describen su principio central, según el cual el usuario declara el estado que desea que el clúster tenga, y un controlador ejecuta de forma permanente un bucle de reconciliación que compara ese estado con el observado y actúa hasta hacerlos converger. Como toda acción se decide a partir de la observación del estado presente, sin depender de una máquina de estados que recuerde por dónde iba el proceso, un controlador que falla y se reinicia retoma el trabajo desde lo que encuentra. Esa propiedad es la que permite operar una infraestructura descrita íntegramente en archivos de configuración versionados y red desplegarla sin reconstruirla a mano, algo que el proyecto necesita porque el clúster se levanta desde cero sobre las estaciones de trabajo de la sala.

Sobre ese modelo se apoya la vía de extensión. Una definición de recurso personalizado (*Custom Resource Definition*, CRD) registra en el servidor de API un tipo de objeto que Kubernetes no conocía, junto con el esquema que valida sus campos, de modo que un concepto propio de un dominio pasa a ser un objeto de primera clase del clúster. El controlador que acompaña a ese tipo de recurso, y que encapsula el conocimiento operativo necesario para materializarlo, recibe el nombre de operador. Xu et al. (2024) estudiaron 210 defectos recolectados de 36 operadores de uso extendido y encontraron que sus fallas más frecuentes provienen del manejo incorrecto del estado durante la reconciliación. De ese hallazgo se sigue una decisión de ingeniería explícita para el proyecto, que consiste en componer operadores ya maduros para todo aquello que alguno de ellos cubra, y reservar el desarrollo propio para la lógica que ninguno provee.

Las extensiones que el proyecto compone cubren tres necesidades. El NVIDIA GPU Operator aprovisiona de forma declarativa el controlador de dispositivo, el *runtime* de contenedores y el complemento que expone la tarjeta gráfica al programador del clúster como recurso asignable, trabajo que de otro modo exigiría una instalación manual por nodo. KubeRay, sobre el *framework* de cómputo distribuido que Moritz et al. (2018) describen a partir de tareas asíncronas y actores con estado, aporta los recursos con los que un clúster de cómputo, un trabajo por lotes y un servicio de inferencia se declaran como objetos nativos, de modo que desplegar un modelo deja de ser la ejecución de un *script* sobre una máquina concreta. Una capa de admisión y encolamiento, por último, retiene los trabajos hasta que el clúster puede garantizarles la capacidad completa que solicitan, lo que evita que varias cargas parcialmente aprovisionadas se bloqueen entre sí; Gao et al. (2022) sitúan ese mecanismo entre las políticas de admisión y expropiación propias de los clústeres de aprendizaje profundo, y es además el punto donde se materializan las políticas de gobernanza de la Apartado 4.5.

4.5. Gobernanza de recursos compartidos

Una infraestructura de cómputo acelerado compartida entre usuarios con necesidades desiguales plantea un problema de asignación que la literatura de sistemas ha abordado desde dos familias con supuestos opuestos. La primera reparte la capacidad de forma proporcional entre solicitantes considerados equivalentes y optimiza alguna métrica agregada del clúster. Xiao et al. (2018) explotan la estructura iterativa de las cargas de aprendizaje profundo para migrar y reempaquetar trabajos en ejecución, Gu et al. (2019) asignan sin conocer de antemano la duración de cada trabajo, y Mahajan et al. (2020), Narayanan et al. (2020) y Qiao et al. (2021) formulan la asignación como un problema de equidad a largo plazo, generalizado a aceleradores heterogéneos y acoplado a los parámetros de cada entrenamiento, sobre la base de sistemas de gestión de clústeres como el que documentan Verma et al. (2015). Lo que todas comparten es el supuesto de que los solicitantes tienen el mismo derecho sobre el recurso.

La segunda familia parte del supuesto contrario, porque establece jerarquías explícitas entre clases de solicitante y garantiza acceso preferente a las superiores mediante prioridad y, cuando hace falta, expropiación de recursos ya asignados. Gao et al. (2022) tratan ambas familias por separado en su taxonomía porque persiguen objetivos incompatibles, equidad entre pares en un caso y garantía de acceso en el otro, de modo que la elección entre ellas responde a una definición organizacional antes que técnica. El contexto académico del IASLab corresponde sin ambigüedad a la segunda. Estudiantes y profesores reservan capacidad, pero los profesores requieren prioridad para reclamar recursos y para controlar el reparto de la capacidad de la sala, incluso por encima de una reserva estudiantil activa, porque la docencia y la investigación dirigida tienen precedencia institucional sobre la exploración individual. Ninguna política de reparto proporcional entre iguales satisface ese requisito, por bien que optimice la utilización agregada.

Dos mecanismos instrumentan la gobernanza que el proyecto necesita. La reserva anticipada aparta capacidad por adelantado para una carga específica durante una ventana de tiempo definida; Weitzel et al. (2025) documentan su introducción sobre unidades de procesamiento gráfico en un clúster Kubernetes multiusuario de escala nacional y reportan un aumento sustancial de la utilización promedio, lo que muestra que la reserva mejora el aprovechamiento del *hardware* al reducir el tiempo que los recursos pasan ociosos entre usos no coordinados. El sobreaprovisionamiento controlado la complementa al asignar más capacidad nominal de la instalada, bajo el supuesto estadístico de que los solicitantes no agotan su cuota máxima de forma simultánea; el problema se ha formalizado como un empaquetado bajo restricciones probabilísticas (Cohen et al., 2019) y desplegado a escala de producción en múltiples centros de datos (Bashir et al., 2021), y su viabilidad descansa sobre la predictibilidad del consumo real frente al declarado, que Cortez et al. (2017) establecen al caracterizar las cargas de una plataforma de nube a gran escala. El margen que el proyecto adopta, entre el 10 % y el 20 %, proviene de la experiencia operativa del laboratorio, sin respaldo directo en estas fuentes.

En el plano de la implementación, las cuotas nativas que Kubernetes asocia a los espacios de nombres acotan el consumo agregado de procesador, memoria y dispositivos de un conjunto de cargas, y constituyen el punto de aplicación sobre el que se construyen la reserva y el margen de sobreasignación. La cuota define cuánto puede consumir un espacio de nombres, y la capa de admisión de la Apartado 4.4 decide en qué momento un trabajo concreto obtiene ese consumo.

Conviene señalar, por último, que gobernar cómputo acelerado en una institución educativa tiene rasgos que la literatura de centros de datos comerciales no captura. George (2020) revisan el uso de unidades de procesamiento gráfico en contextos de aula e investigación y sitúan el problema en un marco donde el objetivo consiste en sostener el acceso de una población estudiantil rotativa, con demandas intermitentes y desiguales, antes que en maximizar el retorno económico por hora de acelerador. Ese encuadre es el que corresponde al IASLab y el que justifica organizar la gobernanza alrededor de reservas y prioridades de rol.

4.6. Identidad federada y control de acceso

Aplicar una política de prioridad por rol exige saber con certeza quién solicita un recurso y qué rol ostenta. El control de acceso basado en roles sigue siendo el modelo de referencia para autorización en sistemas multiusuario, con extensiones para cómputo en la nube que incorporan atributos del solicitante y niveles de confianza a la decisión (Li et al., 2015). La autenticación federada resuelve el problema complementario al delegar la verificación de la identidad en un proveedor externo, que tras autenticar al usuario emite un *token* firmado con sus atributos, entre ellos sus roles; Fett et al. (2016) establecen mediante análisis formal las garantías de autorización, autenticación e integridad de sesión que ofrece OAuth 2.0 bajo ese esquema. La Universidad Icesi opera un proveedor institucional, SAAMFI, comparable en su función con productos como Keycloak o Auth0. Una plataforma apoyada en él hereda su taxonomía de roles y define por encima solo los que su propia operación requiere, como el de administrador, lo que evita comprometerse con una enumeración de perfiles académicos que pertenece al proveedor y puede cambiar sin intervención del proyecto.

La aplicación de cuotas sobre el tráfico de inferencia se concentra, además, en un único punto de entrada, según el patrón que la literatura de microservicios documenta como *API gateway*, un componente que agrega las interfaces de los servicios subyacentes, aplica sobre el tráfico las políticas transversales de autenticación, autorización y límite de tasa, y registra el consumo (Montesi and Weber, 2016). Su variante especializada para modelos de lenguaje añade límites basados en el consumo de *tokens*, que es la unidad en la que se contabiliza el uso de ese tipo de modelos. La revisión de esta literatura no localizó ninguna variante que aplique cuotas de forma unificada sobre cargas de lenguaje y cargas que no lo son, vacío relevante para el proyecto porque la plataforma debe gobernar cualquier carga de inteligencia artificial, y un mecanismo que solo sepa contar *tokens* deja fuera de la política a todo modelo cuya salida no se mida en esa unidad.

4.7. Observabilidad y telemetría de infraestructura acelerada

La observabilidad de un sistema distribuido se define como la capacidad de explicar su comportamiento interno a partir de lo que puede medirse desde afuera, y se apoya en tres señales complementarias. Las métricas agregan magnitudes numéricas a lo largo del tiempo, las trazas siguen el recorrido de una solicitud a través de los componentes que la atienden y los registros conservan los eventos discretos que cada componente emite (Li et al., 2021). Las tres resultan ne-

cesarias porque responden preguntas distintas. Una métrica indica que algo se degradó, una traza indica dónde y un registro indica qué ocurrió en ese punto.

En un clúster de cómputo acelerado compartido, esa capacidad depende de telemetría *fine-grained* de *hardware*, y la evidencia empírica es abundante. Weng et al. (2022) documentan, en un clúster de producción a gran escala, patrones sostenidos de subutilización que solo la instrumentación por nodo permite detectar porque las métricas agregadas los ocultan; Jeon et al. (2019) muestran que una parte sustancial del tiempo total de los trabajos se consume en espera y en fallos, visible únicamente cuando se instrumentan las colas además de los nodos; Gao et al. (2024) encuentran que la mayoría de los casos de baja utilización obedece a causas corregibles identificables a partir de la telemetría; y Liu et al. (2022) muestran que un volumen extenso de telemetría recolectada durante meses eleva de forma sustancial la precisión con que se anticipan los fallos de *hardware*. De este conjunto se desprenden las señales primarias para diagnosticar saturación y anticipar fallos, entre ellas la temperatura del núcleo y de la memoria, la utilización de memoria de video, el consumo de potencia y el *throughput* del bus PCIe.

El ecosistema de Kubernetes instrumenta esas señales con un conjunto de componentes establecido. Prometheus recolecta métricas por consulta periódica a los puntos de exposición de cada componente, las almacena como series temporales identificadas por etiquetas y evalúa sobre ellas reglas de alerta. Loki agrega los registros que emiten los contenedores y los indexa por las mismas etiquetas, lo que permite pasar de una anomalía observada en una gráfica a los registros que la produjeron sin cambiar de sistema de coordenadas. Grafana consolida ambas fuentes en paneles de visualización. El exportador DCGM de NVIDIA completa el arreglo al exponer a Prometheus las métricas de bajo nivel de las tarjetas aceleradoras, que residen en el *firmware* del dispositivo y que ningún otro componente puede obtener.

4.8. Marco conceptual y categorías de análisis

Los conceptos desarrollados en las secciones anteriores se emplean en el resto del documento con el significado operativo acotado que fija la Tabla 4.1, junto con el objetivo específico que cada uno sostiene. La tabla cumple el requisito del formato de relacionar cada concepto definido con los objetivos específicos, y lo concentra en un solo lugar para que la fundamentación de las secciones previas pueda leerse como un argumento continuo.

Cuadro 4.1: *Definición operativa de los conceptos del marco teórico y objetivo específico que los emplea.*

Concepto	Definición operativa en este proyecto	Objetivo
Carga de despliegue	Trabajo que ingresa al clúster a través de esta plataforma y pone en servicio un modelo ya entrenado para atender solicitudes de inferencia	Objetivo específico 3
Motor de inferencia	Componente que carga los pesos de un modelo, gestiona la concurrencia y expone una interfaz de red, empaquetado como imagen de contenedor e intercambiable sin rediseñar la plataforma	Objetivo específico 3

Concepto	Definición operativa en este proyecto	Objetivo
Recurso personalizado y operador	Tipo de objeto registrado en el servidor de API y controlador que lo reconcilia. Es el mecanismo por el que la plataforma declara clústeres de cómputo, trabajos y servicios de inferencia como objetos nativos del orquestador	Objetivo específico 3
Admisión y encolamiento	Retención de un trabajo hasta que el clúster puede garantizarle la capacidad completa que solicita. Es el punto donde se materializa la política de prioridad	Objetivo específico 2
Reserva con prioridad de rol	Asignación anticipada de capacidad por ventana de tiempo, con un margen de sobreasignación del 10 % al 20 % y con potestad de los profesores para reclamar recursos y controlar el reparto de la capacidad de la sala	Objetivo específico 2
Identidad federada	Delegación de la autenticación en SAAMFI, cuyos roles la plataforma hereda y sobre los cuales define un único rol adicional de administración	Objetivo específico 2
Telemetría de nodo	Captura de métricas de <i>hardware</i> y de carga de trabajo por nodo, visualizadas en paneles que muestran el estado de cada nodo del clúster	Objetivo específico 1
Aceptación operacional	Grado en que la plataforma sostiene la concurrencia objetivo de veinte usuarios simultáneos y alcanza un nivel satisfactorio de usabilidad percibida	Objetivo específico 4

Las dos últimas filas fijan las categorías bajo las cuales se analiza el resultado del proyecto, cada una con un instrumento de respaldo metodológico propio. El desempeño se observa a través de las métricas de *hardware* e inferencia que la propia plataforma produce, recogidas bajo pruebas de carga cuyo diseño, ejecución y análisis siguen la sistematización que Jiang and Hassan (2015) elaboran sobre la literatura de pruebas de carga en sistemas de gran escala. La aceptación operacional se observa mediante el *System Usability Scale*, cuestionario de diez ítems cuyas bandas de referencia por ítem recalibraron Lewis and Sauro (2018) a partir de 446 estudios y más de cinco mil respuestas, lo que permite interpretar el puntaje frente a una línea base publicada en lugar de valorarlo de forma aislada. El Capítulo 6 desarrolla la aplicación concreta de ambos dentro de la fase de evaluación empírica.

Capítulo 5

Estado del arte

Capítulo 6

Metodología

Este capítulo formula y justifica la metodología de ingeniería adoptada para resolver el problema planteado en el Capítulo 2, describe la estructura lógica del proceso de investigación, desde la recolección de datos hasta la presentación de resultados, define las fases de desarrollo del proyecto en relación directa con los objetivos específicos del Capítulo 3, presenta el cronograma del proyecto y analiza los riesgos y limitaciones que pueden afectar su alcance, cronograma y calidad.

6.1. Estrategia metodológica

El proyecto adopta un proceso de desarrollo iterativo e incremental de tipo Scrum, siguiendo la *Scrum Guide* de Schwaber and Sutherland (2020), organizado en *sprints* de tres semanas. Esta elección responde a que el alcance del proyecto, cuatro objetivos específicos con dependencias técnicas parciales entre sí, se beneficia de ciclos cortos de construcción y retroalimentación, en los que cada *sprint* entrega un incremento verificable y permite reordenar el trabajo restante conforme avanza el desarrollo y conforme lo permite la disponibilidad de los nodos de cómputo del IAsLab, en lugar de comprometer de antemano una secuencia rígida de actividades como exigiría un modelo de desarrollo en cascada.

La duración de tres semanas por *sprint* responde a la naturaleza del trabajo de las primeras iteraciones. Levantar el clúster comprende una cadena de actividades, entre ellas desplegar Kubernetes sobre las estaciones de trabajo de la sala 104M, configurar la red, el nodo de control y los nodos de trabajo, aprovisionar el acceso a las unidades de procesamiento gráfico y, sobre esa base ya estable, integrar las extensiones que la plataforma necesita. Cada una de ellas depende de que la anterior quede estable, de modo que una iteración más corta obligaría a cerrar *sprints* sin un clúster verificable. El equipo adoptó por eso una cadencia de tres semanas, que permite terminar cada iteración con infraestructura funcionando y comprobable.

Los artefactos y ceremonias que el equipo emplea de forma efectiva son los que exige el curso de proyecto de grado, es decir, un repositorio Jira para el *backlog* y el seguimiento de las iteraciones, un repositorio Bitbucket para el control de versiones y los archivos de configuración de la infraes-

estructura, y una reunión semanal de seguimiento con el tutor del proyecto, en la que se revisa el avance de la iteración en curso y se ajusta el alcance de la siguiente. El curso de Proyecto de Grado I exige, como mínimo, el anteproyecto y dos *sprints*. No se documentan otras ceremonias de Scrum, por ejemplo el *daily scrum* o las retrospectivas formales, porque el equipo no las practica de forma sistemática; el punto de control periódico del proyecto es la reunión semanal con el tutor.

La estructura lógica del proceso de investigación se organiza en cuatro etapas. La recolección de datos se apoya en tres fuentes instrumentadas por el propio sistema, que son la telemetría de *hardware* e inferencia que produce el módulo de observabilidad, los resultados que genera el motor de *benchmarking* automatizado sobre los nodos de inferencia, y los instrumentos de usabilidad aplicados a los usuarios finales de la plataforma. La organización y sistematización de estos datos consiste en consolidar las métricas de *hardware*, de inferencia y de percepción de usuario en formatos comparables entre fases y entre motores de inferencia evaluados. El análisis contrasta estas métricas contra los objetivos específicos formulados, determinando en qué medida cada módulo construido resuelve la carencia que motivó su desarrollo. Finalmente, la interpretación y presentación de resultados traduce los hallazgos del análisis en conclusiones sobre el nivel de aceptación operacional de la plataforma.

La evaluación empírica prevista en objetivo específico 4 se apoya en dos instrumentos con respaldo metodológico propio y en un criterio cuantitativo de aceptación, según el cual el sistema debe soportar 20 usuarios simultáneos utilizando la plataforma. Para el desempeño técnico, el proyecto recurre a pruebas de carga sobre los nodos de inferencia bajo esa concurrencia objetivo; Jiang and Hassan (2015), en su revisión de la literatura sobre pruebas de carga en sistemas de gran escala, sistematizan el diseño de la carga, su ejecución y el análisis de los resultados obtenidos, estructura que este proyecto sigue para contrastar el comportamiento de los motores de inferencia desplegados bajo esa concurrencia realista de usuarios. Para la aceptación operacional por parte de los usuarios finales, se emplea el *System Usability Scale*, un cuestionario de diez ítems utilizado como instrumento estándar para cuantificar la usabilidad percibida de un sistema de *software*. Lewis and Sauro (2018) recalibraron sus bandas de referencia por ítem a partir de 446 estudios y más de cinco mil respuestas individuales, lo que permite interpretar el puntaje obtenido frente a una línea base publicada en lugar de valorarlo de forma aislada. El instrumento se aplica a los distintos roles de usuario del IASLab identificados en el Capítulo 1.

6.2. Fases de desarrollo del proyecto

[TODO: Revisar, si la informacion que da la tabla y el texto es muyh parecida, mejor resuman el texto del apartado y presentan la tabla.] Las fases de desarrollo guardan una relación directa con los objetivos específicos formulados en el Capítulo 3, de modo que cada fase produce el artefacto necesario para satisfacer uno de ellos. El trabajo parte de levantar la infraestructura del clúster desde cero sobre las máquinas de la sala 104M, sin partir de un sistema previo que estabilizar, y el orden de las fases responde a la cadena de dependencias técnicas que se describe a continuación.

La puesta en marcha del clúster ocupa la primera fase porque todo lo demás se ejecuta sobre

él. Los motores de inferencia se despliegan como contenedores que el orquestador programa, la telemetría recolecta métricas de nodos que el clúster administra, y las cuotas se aplican sobre los recursos que el clúster reparte. El equipo deja el clúster desplegado, configurado y sirviendo modelos antes de construir la plataforma web que lo expone, de modo que la interfaz se diseñe sobre capacidades que ya funcionan en lugar de sobre supuestos acerca de cómo funcionarán. Por la misma razón, el proyecto no contempla una fase de prototipado de interfaz previa al desarrollo de la infraestructura.

La observabilidad ocupa la segunda fase porque es el instrumento con el que se mide el efecto de todo lo que se construye después. Sin telemetría por nodo, ni el esquema de cuotas ni la plataforma pueden evaluarse con evidencia, y las pruebas de carga de la última fase quedarían sin datos que interpretar. La gobernanza de cuotas precede a la plataforma web porque su punto de aplicación vive dentro del clúster, en los mecanismos de cuota y de admisión que gobiernan la asignación de recursos, mientras que la plataforma únicamente expone y administra esas políticas; construir la interfaz antes que el mecanismo obligaría a rehacerla cuando el mecanismo quede definido. La plataforma de despliegue y administración ocupa la cuarta fase, ya sobre un clúster instrumentado y gobernado. La evaluación empírica cierra el proceso porque requiere que las cuatro fases anteriores estén construidas para contrastarlas, en conjunto, contra los objetivos que les dieron origen. La Tabla 6.1 resume esta correspondencia y asigna a cada fase una duración en *sprints* de tres semanas.

Cuadro 6.1: *Fases de desarrollo del proyecto, su correspondencia con los objetivos específicos y su duración.*

Fase	Actividades principales	Objetivo específico	Duración
Puesta en marcha del clúster	Despliegue y configuración de Kubernetes sobre las estaciones de trabajo de la sala 104M, aprovisionamiento de las unidades de procesamiento gráfico mediante el NVIDIA GPU Operator, integración de KubeRay y despliegue de los motores de inferencia como imágenes de contenedor	Objetivo específico 3	2 <i>sprints</i>
Observabilidad de infraestructura	Instrumentación de métricas de <i>hardware</i> y de carga de trabajo por nodo con Prometheus, el exportador DCGM, Loki y Grafana	Objetivo específico 1	2 <i>sprints</i>
Gobernanza de cuotas	Implementación del esquema de reserva de cupos y de las restricciones de infraestructura dentro del clúster, con prioridad de los profesores sobre la asignación de la capacidad de la sala y acoplamiento a los atributos de rol de SAAMFI	Objetivo específico 2	2 <i>sprints</i>
Plataforma de despliegue y administración	Construcción del servicio y la interfaz web con los que un usuario despliega modelos entrenados de forma concurrente, y del módulo administrativo que gestiona las políticas de cuota y presenta la telemetría recolectada	Objetivo específico 3, objetivo específico 2	3 <i>sprints</i>

Fase	Actividades principales	Objetivo específico	Duración
Evaluación empírica	Ejecución de pruebas de carga con 20 usuarios simultáneos sobre los nodos de inferencia mediante el motor de <i>benchmarking</i> automatizado, y aplicación del <i>System Usability Scale</i> con usuarios finales	Objetivo específico 4	1 <i>sprint</i>

6.3. Cronograma

El proyecto inicia el 10 de agosto de 2026 y finaliza en mayo de 2027. Entre el 10 de agosto y el 20 de septiembre de 2026 el equipo dedica una etapa previa a la formulación del anteproyecto, el levantamiento de requisitos con el laboratorio y la definición de la arquitectura propuesta. Los *sprints* de desarrollo comienzan el 21 de septiembre de 2026 y se suceden en ciclos de tres semanas. El formato de anteproyecto de la facultad exige que las actividades del cronograma se deriven de los objetivos específicos del Capítulo 3 y sean más detalladas y medibles que ellos. La Tabla 6.2 presenta ese desglose, con las fechas de inicio y fin de cada *sprint* y su entregable. La tabla omite la columna de responsable porque todas las actividades las asume el equipo del proyecto en su conjunto.

[**TODO: Me parece adecuado lo que mencionan, pero recomiendo narrar los aspectos del cronograma sin mencionar las materias de pdg, sino mas bien como el proyecto en si mismo. No relacionarlo a las materias, sus nombres y semestres academicos.**] El proyecto de grado se desarrolla en dos cursos consecutivos. Durante PDG1, entre agosto y diciembre de 2026, se formula el anteproyecto y se entrega a inicios de diciembre de 2026, y en paralelo se ejecutan los cuatro primeros *sprints*, correspondientes a la puesta en marcha del clúster y a la observabilidad de infraestructura. Durante PDG2, entre enero y mayo de 2027, se ejecutan las fases de gobernanza de cuotas, plataforma de despliegue y administración, y evaluación empírica, y el documento del anteproyecto se amplía con el desarrollo realizado y los resultados obtenidos.

Cuadro 6.2: Cronograma del proyecto, por sprints de tres semanas, con entregables asociados.

<i>Sprint</i>	Actividad	Fechas	Entregable
Etapa previa	Formulación del anteproyecto, levantamiento de requisitos con el laboratorio y definición de la arquitectura propuesta	10 ago – 20 sep 2026	Documento de arquitectura y especificación de requisitos
<i>Sprint 1</i>	Despliegue de Kubernetes sobre las estaciones de trabajo de la sala 104M, con configuración de red, nodo de control y nodos de trabajo	21 sep – 11 oct 2026	Clúster Kubernetes operativo y su configuración versionada

<i>Sprint</i>	Actividad	Fechas	Entregable
<i>Sprint 2</i>	Aprovisionamiento de las GPU mediante el NVIDIA GPU Operator, integración de KubeRay y despliegue de los motores de inferencia como imágenes de contenedor	12 oct – 1 nov 2026	Modelo desplegado y consumible sobre el clúster
<i>Sprint 3</i>	Instrumentación de métricas de <i>hardware</i> por nodo (temperatura, VRAM, consumo energético, PCIe) y agregación centralizada de registros	2 nov – 22 nov 2026	Prometheus, exportador DCGM y Loki desplegados en el clúster
<i>Sprint 4</i>	Instrumentación de métricas de inferencia y construcción de los paneles de visualización	23 nov – 13 dic 2026	Panel de telemetría en Grafana (objetivo específico 1)
Hito	Entrega del anteproyecto (cierre documental de PDG1)	Inicios de dic. 2026	Documento de anteproyecto
<i>Sprint 5</i>	Implementación del esquema de reserva de cupos y de las restricciones de recursos dentro del clúster	11 ene – 31 ene 2027	Mecanismo de cuotas operativo en el clúster
<i>Sprint 6</i>	Acoplamiento de las cuotas a los atributos de rol de SAAMFI y prioridad de los profesores sobre la capacidad de la sala	1 feb – 21 feb 2027	Sistema de gobernanza por rol (objetivo específico 2)
<i>Sprint 7</i>	Construcción del servicio de la plataforma que expone el despliegue de modelos sobre el clúster	22 feb – 14 mar 2027	Servicio de despliegue de modelos
<i>Sprint 8</i>	Interfaz web de despliegue concurrente de modelos, con selección de nodo y de motor de inferencia	15 mar – 4 abr 2027	Interfaz de despliegue de modelos (objetivo específico 3)
<i>Sprint 9</i>	Módulo administrativo de gestión de políticas de cuota y presentación de la telemetría dentro de la plataforma	5 abr – 25 abr 2027	Módulo administrativo de la plataforma
<i>Sprint 10</i>	Ejecución del motor de <i>benchmarking</i> automatizado bajo carga de 20 usuarios simultáneos y aplicación del <i>System Usability Scale</i>	26 abr – 16 may 2027	Reportes de pruebas de carga y de usabilidad (objetivo específico 4)
Cierre	Ampliación del documento del anteproyecto con el desarrollo realizado y los resultados obtenidos, y redacción del informe final	17 may – 31 may 2027	Informe final de PDG2

6.4. Análisis de riesgos y limitaciones

[**TODO: Esa primera frase, qué?**] Esta sección ubica el análisis de riesgos y limitaciones dentro del capítulo de metodología porque así lo dispone el formato de anteproyecto de la facultad. Los riesgos que se describen a continuación se derivan de las restricciones y los supuestos del proyecto, se enfocan en el nivel técnico y de proceso, y se presentan junto con la dimensión del proyecto que comprometen y la estrategia prevista para mitigarlos.

[**TODO: Si hacen referencia al objetivo específico 2, que eso me lleve a la seccion, poner hipervinculo. En un apartado habla sobre FR03.4, pero no especifica qué es eso o a qué hace referencia.**]

Falta de disponibilidad de los nodos de cómputo del IAsLab Compromete el cronograma, dado que las fases de desarrollo y, en particular, la evaluación empírica dependen del acceso continuo a los nodos de cómputo del laboratorio, condición que el proyecto asume vigente pero no controla directamente. Como estrategia de mitigación, las actividades de desarrollo que no requieren *hardware* de inferencia (por ejemplo, la definición del esquema lógico de cuotas) se priorizan en los periodos de menor disponibilidad de los nodos.

Cambios o caída del servicio SAAMFI SAAMFI es el proveedor de identidad institucional de la Universidad Icesi, que emite un *token* con los roles y permisos del usuario, y cada sistema que lo integra decide cómo usar esos atributos. Este riesgo compromete el alcance y el cronograma, pues el control de acceso y la gobernanza de cuotas por rol dependen de los atributos que provee ese servicio. La mitigación consiste en aislar la lógica de mapeo de roles en un componente propio de la plataforma, de modo que un cambio en la interfaz de SAAMFI exija ajustar solo ese componente, sin rediseñar el esquema de cuotas completo.

Saturación de VRAM y congelamiento de nodos GPU Compromete la calidad del servicio de inferencia y, potencialmente, el cronograma de las pruebas de carga. El equipo del laboratorio ha confirmado que un uso sostenido de la GPU entre 90 % y 95 % congela las máquinas de la sala, y que la causa física es que la fuente de poder de la torre no soporta el consumo eléctrico resultante. Liu et al. (2022) documentan que este tipo de fallos es predecible con telemetría suficiente en producción, por lo que la mitigación prevista es que el módulo de observabilidad (objetivo específico 1) genere las alertas de diagnóstico automatizado que exige FR-03.4 antes de que la saturación derive en un congelamiento no controlado.

Uso simultáneo de los equipos para actividades académicas Los equipos de la sala deben seguir siendo usables para las clases mientras el clúster corre sobre ellos, restricción operativa del laboratorio. Compromete la calidad del servicio de inferencia, en tanto la carga docente puede competir por los mismos recursos de cómputo que utiliza la plataforma. La mitigación consiste en que el esquema de cuotas (objetivo específico 2) reserve capacidad para las cargas del proyecto sin bloquear el uso académico habitual de los equipos, y en ejecutar fuera del horario de clases toda prueba que pueda degradar el rendimiento de las máquinas.

Cambios de requisitos que obliguen a rehacer la infraestructura El tutor advirtió que, al terminar el proyecto, pueden surgir necesidades nuevas que obliguen a reemplazar buena parte de lo construido. Compromete el alcance y el cronograma de mantenimiento posterior al

proyecto. La mitigación es el compromiso adquirido por el equipo con GitOps, documentación y automatización de la infraestructura, de modo que un cambio de requisitos permita redespigar o adaptar el clúster (por ejemplo, en otra sala del laboratorio) sin reconstruirlo desde cero.

Límites de ancho de banda de la sala Compromete la calidad de las pruebas de carga, en tanto la red de la sala del IASLab tiene una capacidad de 10 Gbit/s. El proyecto no requiere una capacidad mayor, porque no contempla inferencia distribuida entre nodos por red. La mitigación consiste en diseñar los escenarios de prueba de carga a partir de esa capacidad real, en lugar de extrapolar cifras de referentes de la literatura obtenidos en infraestructuras de mayor escala.

Capítulo 7

Contribución y resultados del proyecto de grado

7.1. Aportes relacionados con el objeto del proyecto

La plataforma descrita en los capítulos anteriores incide directamente sobre el problema formulado en Capítulo 2, que reúne la fragmentación del ciclo de vida de MLOps y el riesgo de uso inequitativo de la infraestructura del IAsLab. Su primer aporte es de naturaleza institucional y económica. Al habilitar el despliegue distribuido de modelos ya entrenados sobre el mismo *hardware* que hoy se usa para el entrenamiento, el proyecto maximiza el retorno de la inversión realizada en la infraestructura del laboratorio, en lugar de dejarla limitada a la fase experimental del ciclo de vida. El sistema de reserva de cupos con prioridad por rol, acoplado a los roles institucionales, amplía el acceso a las unidades de procesamiento gráfico entre estudiantes y profesores de la universidad, y da a los profesores prioridad para reclamar recursos y controlar el reparto de la capacidad de la sala. Esta misma capacidad reduce la necesidad de adquirir créditos de cómputo en nubes públicas comerciales para sostener cargas de trabajo que la infraestructura *on-premise* del laboratorio puede absorber, argumento que se desarrolla en Capítulo 1.

En el plano académico, el proyecto habilita la ejecución de proyectos avanzados en áreas prioritarias para el macroproyecto, como Industria 4.0 y e-Salud, al permitir que arquitecturas de gran escala, en particular *Large Language Models*, pasen de la fase de entrenamiento a un servicio de inferencia consumible dentro de la misma plataforma institucional. Esta capacidad tiene además una implicación de privacidad y gobernanza de datos, porque al mantener el procesamiento de inferencia sobre servidores propios del IAsLab, en lugar de delegarlo en proveedores externos, la información asociada a estos proyectos de investigación permanece bajo el control administrativo de la universidad. Finalmente, en el plano operativo, el módulo de telemetría y la administración centralizada de cuotas reducen la carga cognitiva y administrativa que hoy recae sobre los administradores del laboratorio, quienes actualmente carecen de visibilidad *fine-grained* sobre el consumo real de la infraestructura para diagnosticar saturaciones o conflictos de uso entre usuarios.

7.2. Aportes relacionados con el desarrollo de capacidades del investigador

La ejecución de este proyecto exige y desarrolla competencias profesionales específicas de la disciplina, propias del programa de Ingeniería Telemática. El despliegue y la configuración del clúster de Kubernetes sobre las estaciones de trabajo del laboratorio, la integración de KubeRay y del NVIDIA GPU Operator, y la definición de reglas de selección de nodos y aislamiento de cargas de trabajo desarrollan competencias de orquestación de contenedores y cómputo distribuido en topologías heterogéneas de CPU y GPU. La integración de motores de inferencia empaquetados como imágenes de contenedor, elegidos por medición y con una compilación de *llama.cpp* ajustada al *hardware* de la sala como candidato preferente para servir modelos ya cuantizados en formato GGUF dentro de la memoria de video disponible, desarrolla, adicionalmente, competencias de ingeniería de plataformas de inferencia bajo restricciones de *hardware*.

La implementación del sistema de reserva de cupos con prioridad por rol, acoplado a los atributos de rol expuestos por el servicio de identidad institucional SAAMFI, exige competencias de diseño de esquemas de gobernanza y de integración con sistemas de control de acceso basado en roles (RBAC), un dominio propio de la ingeniería de *software* orientada a sistemas multiusuario. La construcción del módulo de observabilidad, apoyado en Prometheus, Grafana, Loki y el exportador DCGM de NVIDIA, desarrolla competencias de ingeniería de observabilidad de infraestructura distribuida, entendida como la capacidad de instrumentar, recolectar y visualizar métricas de *hardware* e inferencia en tiempo real. Por último, la evaluación de la plataforma mediante pruebas de carga sobre los nodos de inferencia, un motor de *benchmarking* automatizado y validaciones de usabilidad con usuarios reales desarrolla competencias de diseño y ejecución de evaluación empírica de sistemas de *software*, indispensables para sustentar decisiones de ingeniería con evidencia.

7.3. Resultados y entregables

De acuerdo con el formato institucional del anteproyecto, cada resultado esperado y su entregable correspondiente deben trazarse a al menos uno de los objetivos específicos formulados en Capítulo 3. La Tabla 7.1 presenta esta trazabilidad para los cuatro objetivos específicos del proyecto.

Cuadro 7.1: Trazabilidad entre objetivos específicos, resultados esperados y entregables.

Objetivo específico	Resultado esperado	Entregable
Objetivo específico 1: desarrollo del módulo de observabilidad	Panel de telemetría que muestra, por nodo, métricas de <i>hardware</i> (temperatura, VRAM, consumo energético) y de carga de trabajo (latencia, <i>tokens</i> por segundo, saturación de concurrencia), mediante visualizaciones gráficas interactivas	Módulo de observabilidad integrado a la plataforma (Prometheus, DCGM Exporter, Grafana, Loki) y su documentación de despliegue

Objetivo específico	Resultado esperado	Entregable
Objetivo específico 2: implementación del sistema de cuotas por rol	Sistema de cuotas y restricciones de infraestructura que reserva cupos de cómputo para las cargas de inferencia, con prioridad de los profesores sobre la asignación de la capacidad de la sala, acoplado a los roles provistos por SAAMFI	Módulo administrativo de gestión de cuotas integrado a la plataforma web y manual de administración de políticas de cuotas
Objetivo específico 3: habilitación del despliegue de modelos	Clúster de cómputo que ejecuta motores de inferencia intercambiables, empaquetados como imágenes de contenedor, sobre el cual un usuario despliega modelos ya entrenados de forma concurrente desde la plataforma	Clúster desplegado con su configuración versionada, interfaz de despliegue de modelos integrada a la plataforma y manual de despliegue
Objetivo específico 4: evaluación de desempeño y aceptación	Reporte de desempeño de la plataforma bajo pruebas de carga con veinte usuarios simultáneos sobre los nodos de inferencia, y de aceptación operacional mediante el <i>System Usability Scale</i> aplicado a usuarios del laboratorio	Informe de resultados de pruebas de carga y usabilidad, con los datos generados por el motor de <i>benchmarking</i> automatizado

Los cuatro entregables específicos de la Tabla 7.1 convergen en los dos entregables globales exigidos para el proyecto de grado. El primero es el *software* funcional y empaquetado, es decir, el código fuente de la plataforma, desplegado sobre la infraestructura propia del IAsLab y estructurado bajo estándares que permitan su registro como producto tecnológico con derechos de autor. El segundo es la documentación técnica y de usuario, compuesta por los manuales de despliegue de modelos, de visualización de telemetría y de administración de políticas de cuotas que se derivan de cada uno de los módulos anteriores. Ambos entregables globales dependen del cumplimiento conjunto de los cuatro objetivos específicos, porque ningún módulo constituye, por sí solo, la plataforma que exige el objeto del proyecto.

Bibliografía

- Ahmed, N., Wahed, M., and Thompson, N. C. (2023). The growing influence of industry in AI research. *Science*, 379(6635):884–886.
- Bashir, N., Deng, N., Rzađca, K., Irwin, D. E., Kodak, S., and Jnagal, R. (2021). Take it to the limit: Peak prediction-driven resource overcommitment in datacenters. In *Proceedings of the Sixteenth European Conference on Computer Systems (EuroSys '21)*. ACM.
- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). Borg, omega, and kubernetes: Lessons learned from three container-management systems over a decade. *ACM Queue*, 14(1):70–93.
- Carrión, C. (2022). Kubernetes as a standard container orchestrator: A bibliometric analysis. *Journal of Grid Computing*, 20:42.
- Chen, Y.-D., Zheng, K.-J., and Guo, Z.-H. (2026). A survey of quantization in LLM: Unlocking potential hardware efficiency. *Journal of Computer Science and Technology*, 41(1):341–358.
- Cohen, M. C., Keller, P. W., Mirrokni, V., and Zadimoghaddam, M. (2019). Overcommitment in cloud services: Bin packing with chance constraints. *Management Science*, 65(7):3255–3271.
- Cortez, E., Bonde, A., Muzio, A., Russinovich, M., Fontoura, M., and Bianchini, R. (2017). Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP 17)*, pages 153–167. ACM.
- Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. Curran Associates, Inc.
- Eken, B., Pallewatta, S., Tran, N. K., Tosun, A., and Babar, M. A. (2026). A multivocal review of MLOps practices, challenges and open issues. *ACM Computing Surveys*, 58(2):1–35.
- Fett, D., Küsters, R., and Schmitz, G. (2016). A comprehensive formal security analysis of OAuth 2.0. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*, pages 1204–1215. ACM.
- Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*.

- Gao, W., Hu, Q., Ye, Z., Sun, P., Wang, X., Luo, Y., Zhang, T., and Wen, Y. (2022). Deep learning workload scheduling in GPU datacenters: Taxonomy, challenges and vision. Preprint arXiv:2205.11913.
- Gao, Y., He, Y., Li, X., Zhao, B., Lin, H., Liang, Y., Zhong, J., Zhang, H., Wang, J., Zeng, Y., Gui, K., Tong, J., and Yang, M. (2024). An empirical study on low GPU utilization of deep learning jobs. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE 2024)*. ACM.
- George, N. (2020). Literature review and implementation overview: High performance computing with graphics processing units for classroom and research use. *arXiv preprint arXiv:2005.07598*.
- Gu, J., Chowdhury, M., Shin, K. G., Zhu, Y., Jeon, M., Qian, J., Liu, H., and Guo, C. (2019). Tiresias: A GPU cluster manager for distributed deep learning. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 485–500. USENIX Association.
- Jeon, M., Venkataraman, S., Phanishayee, A., Qian, J., Xiao, W., and Yang, F. (2019). Analysis of large-scale multi-tenant GPU clusters for DNN training workloads. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association.
- Jiang, Z. M. and Hassan, A. E. (2015). A survey on load testing of large-scale software systems. *IEEE Transactions on Software Engineering*, 41(11):1091–1118.
- Kreuzberger, D., Kühn, N., and Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP 23)*. ACM.
- Lewis, J. R. and Sauro, J. (2018). Item benchmarks for the system usability scale. *Journal of Usability Studies*, 13(3):158–167.
- Li, B., Peng, X., Xiang, Q., Wang, H., Xie, T., Sun, J., and Liu, X. (2021). Enjoy your observability: An industrial survey of microservice tracing and analysis. *Empirical Software Engineering*, 27:25.
- Li, H., Wang, S., Tian, X., Wei, W., and Sun, C. (2015). A survey of extended role-based access control in cloud computing. In *Proceedings of the 4th International Conference on Computer Engineering and Networks (CENet 2014)*, volume 355 of *Lecture Notes in Electrical Engineering*. Springer.
- Lima, A., Monteiro, L., and Furtado, A. P. (2022). MLOps: Practices, maturity models, roles, tools, and challenges – a systematic literature review. In *Proceedings of the 24th International Conference on Enterprise Information Systems (ICEIS 2022)*, pages 308–320. SciTePress.
- Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. (2024). AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of the 7th Conference on Machine Learning and Systems (MLSys 2024)*. Best Paper Award.

- Liu, H., Li, Z., Tan, C., Yang, R., Cao, G., Liu, Z., and Guo, C. (2022). Prediction of GPU failures under deep learning workloads. *arXiv preprint arXiv:2201.11853*.
- Mahajan, K., Balasubramanian, A., Singhvi, A., Venkataraman, S., Akella, A., Phanishayee, A., and Chawla, S. (2020). Themis: Fair and efficient GPU cluster scheduling. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association.
- Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Jin, H., Chen, T., and Jia, Z. (2025). Towards efficient generative large language model serving: A survey from algorithms to systems. *ACM Computing Surveys*, 58(1):1–37.
- Montesi, F. and Weber, J. (2016). Circuit breakers, discovery, and API gateways in microservices. Preprint arXiv:1609.05830.
- Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association.
- Muiruri, D., Lwakatare, L. E., Nurminen, J. K., and Mikkonen, T. (2026). Perspectives on machine learning inference serving in real-world settings. *Software: Practice and Experience*.
- Narayanan, D., Santhanam, K., Kazhamiaka, F., Phanishayee, A., and Zaharia, M. (2020). Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association.
- Qiao, A., Choe, S. K., Subramanya, S. J., Neiswanger, W., Ho, Q., Zhang, H., Ganger, G. R., and Xing, E. P. (2021). Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association.
- Schwaber, K. and Sutherland, J. (2020). The scrum guide: The definitive guide to scrum: The rules of the game.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*. Curran Associates, Inc.
- Sevilla, J., Heim, L., Ho, A., Besiroglu, T., Hobbhahn, M., and Villalobos, P. (2022). Compute trends across three eras of machine learning. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE.
- Shamim, S. I., Gibson, J. A., Morrison, P., and Rahman, A. (2022). Benefits, challenges, and research topics: A multi-vocal literature review of kubernetes. Preprint arXiv:2211.07032.
- Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., and Wilkes, J. (2015). Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys 15)*. ACM.

- Weitzel, D., Graves, A., Albin, S., Zhu, H., Würthwein, F., Tatineni, M., Mishin, D., Graham, J., Khoda, E. E., Sada, M. F., Smarr, L., and DeFanti, T. (2025). The national research platform: Stretched, multi-tenant, scientific Kubernetes cluster. In *Practice and Experience in Advanced Research Computing (PEARC '25)*. ACM. Preprint arXiv:2505.22864; verificar paginación final de las actas antes de la entrega.
- Weng, Q., Xiao, W., Yu, Y., Wang, W., Wang, C., He, J., Li, Y., Zhang, L., Lin, W., and Ding, Y. (2022). MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters. In *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 945–960. USENIX Association.
- Xiao, W., Bhardwaj, R., Ramjee, R., Sivathanu, M., Kwatra, N., Han, Z., Patel, P., Peng, X., Zhao, H., Zhang, Q., Yang, F., and Zhou, L. (2018). Gandiva: Introspective cluster scheduling for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association.
- Xu, K., Sun, D., Wang, H., Ren, Z., Wan, X., Liao, X., Wang, Z., Zhang, J., and Chen, K. (2025). Design and operation of shared machine learning clusters on campus. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2025), Volume 1*. ACM. Preprint available as arXiv:2110.01556.
- Xu, Q., Gao, Y., and Wei, J. (2024). An empirical study on kubernetes operator bugs. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1746–1758.
- Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. (2022). Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association.